Information Fusion 125 (2026) 103450

Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus

Enhancing vulnerability detection by fusing code semantic features with LLM-generated explanations

Zhenzhou Tian [a,b,c,*], Minghao Li [a], Jiaze Sun [a,b,c], Yanping Chen [a,b,c], Lingwei Chen [d]

[a] Department of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[b] Shaanxi Key Laboratory of Network Data Analysis and Intelligent Processing, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[c] Xi'an Key Laboratory of Big Data and Intelligent Computing, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[d] Department of Computer Science and Engineering, Wright State University, Dayton, 45435, OH, USA

ARTICLE INFO

Keywords:
Vulnerability detection
Multi-modal fusion
Large language model
Pre-trained language model

ABSTRACT

The rising prevalence of software vulnerabilities highlights the critical need for more effective detection techniques. Although recent deep learning (DL) approaches have made notable progress, they primarily rely on code-centric modalities, such as token sequences, abstract syntax trees (ASTs), and graph-based representations, while largely neglecting complementary semantic cues available from natural language artifacts like code explanations. This paper proposes FuSEVul, a novel multi-modal framework that integrates code semantics with automatically generated natural language explanations to enhance vulnerability detection. FuSEVul consists of three key components: (1) a code semantic encoder that leverages pre-trained model to capture structural and semantic features from source code; (2) an explanation generation module that prompts a large language model (LLM) to produce functional and risk-aware textual descriptions, subsequently encoded using RoBERTa; and (3) a self-attention-based fusion mechanism that dynamically aligns and integrates cross-modal features, emphasizing signals most indicative of vulnerabilities. Extensive experimental evaluations across three public datasets demonstrate that FuSEVul outperforms 18 state-of-the-art baselines, yielding average relative gains of 7.3% in accuracy and 52.2% in F1 score. Ablation studies further confirm the effectiveness of incorporating LLM-generated explanations and the proposed fusion strategy.

1. Introduction

Despite significant efforts to enhance software quality, the growing prevalence of vulnerabilities remains a critical concern in today's digital landscape. By 2025, the Common Vulnerabilities and Exposures (CVE) [1] database has surpassed 270,000 recorded vulnerabilities, with a significant surge of over 40,000 new entries in 2024, marking a concerning 38% increase compared to the quantities reported in 2023. This rapid rise in reported vulnerabilities has been mirrored by a surge in the financial impact of cyberattacks, with the average cost of data breaches reaching a record 4.45 million USD in 2023 [2], according to IBM. This increasing frequency and cost of security incidents underscore the urgent need for effective detection methods.

Witnessed of the remarkable performance of deep learning (DL) in various software analysis tasks [3], numerous DL-based vulnerability prediction approaches have emerged [4]. These methods utilize deep neural encoders, such as Convolutional Neural Networks (CNNs) [5], Graph Neural Networks (GNNs) [6], and pre-trained language models (PLMs) [7], to extract informative features that capture code semantics and vulnerable patterns, from different code views such as token sequence [8], AST [9], and PDG [10]. Considering that leveraging multi-modal information has proven to be an effective strategy for improving feature learning across various tasks [11–13], one technical paradigm in vulnerability prediction focuses on either extracting and fusing diverse features gathered from multiple distinct code views or, alternatively, operating on a single yet more informative code view. For instance, DeepVD [14] first distills features from the token sequence, AST, Post-Dominator Tree (PDT), and Exception Flow Graph (EFG), and then concatenates them for improved vulnerability detection performance; while Reveal [6] uses GGNN to extract indicative features from the more informative Code Property Graph (CPG), which integrates AST, CFG and DFG into one unified graph representation.

Nevertheless, most existing methods treat the code itself as the sole source for extracting vulnerability-indicative features. While the code directly carries vulnerabilities, additional modality data, such

* Corresponding author at: Department of Computer Science and Technology, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China.
E-mail addresses: tianzhenzhou@xupt.edu.cn (Z. Tian), orangeli@stu.xupt.edu.cn (M. Li), sunjiaze@xupt.edu.cn (J. Sun), chenyp@xupt.edu.cn (Y. Chen), lingwei.chen@wright.edu (L. Chen).

https://doi.org/10.1016/j.inffus.2025.103450
Received 6 May 2025; Received in revised form 9 June 2025; Accepted 20 June 2025
Available online 2 July 2025

as natural language comments or textual descriptions that convey the code's purpose, functionality, and other key details, also provide valuable semantic insights. Thus, integrating this auxiliary information into vulnerability detection models, alongside the code itself, may further enhance the model's ability to perceive deeply hidden and subtle vulnerability patterns more effectively.

To this end, this paper follows the multi-modal fusion paradigm and introduces FuSEVul, which **Fu**ses the **S**emantic features extracted from both the code and its natural language **E**xplanations of functionality to enhance **Vul**nerability detection performance. Specifically, FuSEVul is designed with three key modules to effectively capture and emphasize vulnerability-indicative features: (1) **Code Semantic Encoding**, where a pre-trained code model is used to encode the semantic features of the source code by operating on its normalized code token sequence; (2) **Code Explanation Generation and Encoding**, where a large language model (LLM) is prompted to generate informative explanations regarding code functionality and potential risks, and RoBERTa is employed to extract semantic encodings from these explanations; and (3) **Feature Fusion**, where a self-attention-based fusion mechanism is employed to effectively integrates the feature encodings of both the code and its corresponding explanations.

The primary contributions of this work are summarized as follows:

- We present FuSEVul, an new multi-modal vulnerability detection model that offers improved detection capabilities. Unlike traditional approaches that generally operate on a single or multiple code views derived from the code itself, FuSEVul explores integrating natural language code explanation to empower the model more comprehensively and effectively recognizing the hidden vulnerability patterns, for the first time.
- To ensure the natural language explanation be effectively integrated into the model, (1) we prompt an advanced LLM to obtain high quality code explanations that encapsulate both its functionality and potential security risks; (2) we devise a self-attention based fusion strategy to facilitate more adequate cross-modal interactions between the features encoded from the code and its explanations, allowing the model better attending to the relevant aspects of each modality.
- Extensive experimental evaluations on three public datasets demonstrate that FuSEVul achieves superior performance, with an average relative improvement of 7.3% in accuracy and 52.2% in F1 score over the comparison methods. Ablation studies confirm the critical role of its core components, validating their effectiveness in enhancing detection performance. To facilitate relevant researches, we have open-sourced the implementation at https://github.com/XUPT-SSS/FuSEVul.

The remainder of this paper is structured as follows. Section 2 discusses closely related works. Section 3 presents the design details of FuSEVul. The experimental setup is described in Section 4, followed by experimental evaluations and analysis in Section 5. Section 6 explores potential threats to validity, current limitations, and future research directions. Finally, Section 7 concludes this work.

2. Related work

We categorize existing deep learning-based vulnerability prediction approaches into three main groups: supervised learning-based, pre-trained model-based, and LLM-based ones.

2.1. Supervised-based methods

The earlier studies predominantly adopt a conventional supervised learning paradigm, training vulnerability detection models from scratch. Depending on the form of code representation utilized as input to the feature extraction models, these approaches can be broadly categorized into sequence-based and graph-based methods.

The sequence-based approaches [15–17] represent input code snippets as linear token sequences, typically obtained through either basic lexical parsing [16] or more targeted techniques such as control- and data-dependence based code slicing [17,18]. These token sequences are subsequently processed by neural encoders, such as CNNs [19] and LSTMs [17], to extract semantic features and predict the presence of vulnerabilities.

The graph-based approaches [6,20–23] rely on structurally rich graph representations, including AST [9], PDG [24], and more hybrid structures such as Code Property Graphs (CPG) [25], HeVG [10], and MPG [26]. These graph representations are processed by specialized neural architectures, including tree-structured networks or graph neural networks (GNNs), to extract relevant code features for vulnerability prediction. While these methods generally outperform sequence-based approaches in terms of detection accuracy, they face significant challenges related to training efforts, as constructing complex graph structures and training high-performing GNNs is more resource-intensive.

Besides the above approaches relying on unimodal code representations, several studies have explored the integration of features from multiple complementary code views to enhance vulnerability detection performance [27]. For instance, S^2FVD extracts features from token sequence, AST, and CFG using DPCNN, ERvNN, and GAT respectively, and subsequently fuses them through a multilayer perceptron based fusion layer. Similarly, DeepVD [14] incorporates features derived from token sequences, ASTs, Post-Dominator Trees (PDT), and Exception Flow Graphs (EFG), which are concatenated to form a unified feature space for prediction. However, these methods uniformly operate on structural or semantic views extracted directly from the code itself. In contrast, FuSEVul is the first to explore the integration of natural language-based code explanations, offering an orthogonal and language-agnostic modality that enables the model to more effectively capture latent vulnerability patterns beyond what is inferable from code structure alone.

2.2. Pre-trained model-based methods

With the widespread adoption of pre-trained language models [28–31] (PLMs) in program analysis, an increasing number of recent studies have adopted these models as foundational backbones, either by fine-tuning them [7,8,32,33] or reprogramming them [34,35] to adapt to the task of vulnerability detection. Compared to fully supervised models trained from scratch, PLM-based approaches, which are pre-trained on large-scale and heterogeneous code corpora, offer a significantly stronger initialization. This advantage facilitates the training of more effective models, particularly in data-scarce scenarios.

For example, LineVul [7] feeds code token sequences along with positional embeddings into CodeBERT for vulnerability prediction. EPVD [33] extracts syntactic paths from syntax-CFGs and encodes them using CodeBERT and CNNs. SCL-CVD [36] applies supervised contrastive learning to fine-tune GraphCodeBERT using vulnerable-patched code pairs, thereby enhancing the model's ability to recognize vulnerability-indicative patterns. While these fine-tuning strategies involve updating most or all parameters of the underlying PLMs, CAPTURE [34] pioneers an adversarial reprogramming paradigm that minimizes computational cost. It repurposes a pre-trained vision model for vulnerability detection by transforming token sequences into perturbation images and employing dynamic label remapping. Since no parameter of the backbone PLM but merely the perturbation elements are updated during model training, the computational cost and data requirement are largely reduced. However, these works essentially operate on the uni-modal token sequence derived solely from the code; while our method is orthogonal, where we can use these PLMs for better code encoding, but further incorporate textual code explanations as an additional modality to enhance the model's vulnerability detection capability.

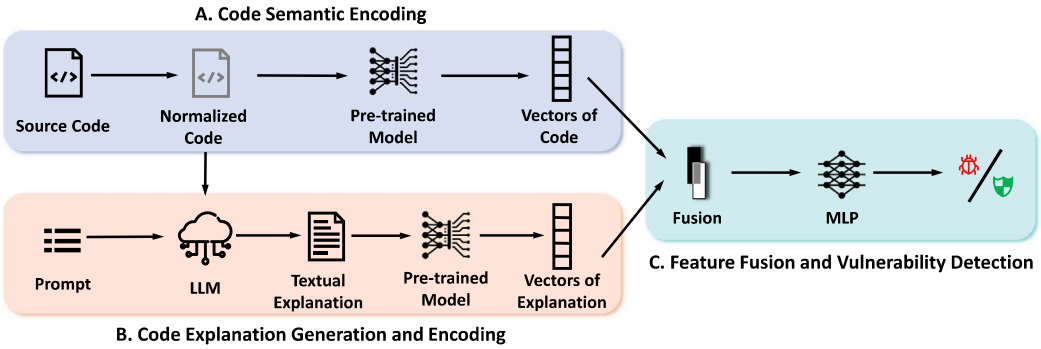

Fig. 1. The overall architecture of FuSEVul.

2.3. LLM-based methods

Large language models (LLMs) have demonstrated encouraging performance across a spectrum of code intelligence tasks, such as code generation [37], code summarization [38], and program translation [39]. In parallel, an emerging line of research [40–45] has begun to investigate the application of LLMs, such as ChatGPT and CodeLLaMA, to the task of vulnerability detection via prompt engineering.

However, there remains no clear consensus regarding the effectiveness of LLMs in this domain. A substantial number of studies [40,42,43,46–48] report that LLMs, when directly prompted, underperform compared to dedicated models. For instance, Yin et al. [41] find that state-of-the-art models trained from scratch or fine-tuned from PLMs consistently surpass LLMs in vulnerability detection. These limitations are often attributed to the difficulty LLMs face in grasping critical program structures and security-centric concepts [49].

Conversely, several studies [44,45,50] demonstrate that, with appropriate prompting strategies, LLMs can achieve competitive or even superior performance relative to conventional baselines. Notably, GRACE [44] illustrates that the integration of similar demonstration examples and code structure information to enforce context-rich prompting can achieve improved detection performance. Techniques such as chain-of-thought (CoT) reasoning [51] and retrieval-augmented generation (RAG) [52] have also shown potential with incorporating richer semantic and contextual cues.

In contrast to these work that directly prompts LLMs for vulnerability prediction, FuSEVul adopts an indirect yet more stable approach by utilizing LLM-generated natural language explanations of code as an auxiliary modality. This strategy builds upon one of the most mature and well-established capabilities of LLMs, i.e., code explanation or comment generation, and effectively complements code-based representations to enhance the detection of latent vulnerability patterns.

3. Method

Fig. 1 illustrates the overall architecture of FuSEVul, which is structured into three principal modules: (1) **Code Semantic Encoding**, in which the raw source code is normalized and represented as a code token sequence, from which a pre-trained code model is used to extract the code semantic features; (2) **Code Explanation Generation and Encoding**, where a LLM is prompted to produce explanations that articulate the code's functionality and potential security risks, and the RoBERTa model is employed to encode the textual explanations; and (3) **Feature Fusion and Vulnerability Prediction**, in which the semantic features distilled from both the source code and its corresponding explanations are integrated through a self-attention based fusion strategy, followed by a MLP-based classification layer to perform the final vulnerability prediction.

3.1. Code semantic encoding

The raw source code may unintentionally or deliberately include identifiers that convey misleading or overly explicit hints, leading the model to learn only superficial patterns for vulnerability detection. For instance, many samples in the TrVD dataset basically have their function or identifier names embedded with vulnerability-related cues, such as "good" and "bad". A specific sample is shown in Fig. 2, where the function name "good" clearly suggests that the sample is benign. Thus, similar to existing methods [5,9], we replace each user-defined function and variable name with unified placeholders as "FUN_i" and "VAR_i", where i denotes their first occurrence order in the code snippet. This reduces the risk of the model simply relying on these indicative names for vulnerability identification. With simplified token space, the impact of out-of-vocabulary (OOV) tokens – like custom identifiers – which can otherwise introduce noise and compromise the model's generalization ability, can also be mitigated.

In addition, sine FuSEVul incorporates semantic information from LLM-generated natural language explanations of the code into the model, pre-existing comments in the code, which may contain suggestive or intentionally misleading information, can either lead to information leakage to the LLM (potentially inflating model performance) or cause the LLM to generate misleading explanations. For instance, the "Fix" comment in Fig. 2 explicitly suggests that the function has been repaired and is no longer vulnerable. To mitigate these risks, we remove all comments from the code.

Given the normalized code snippet, FuSEVul utilizes Byte Pair Encoding (BBPE) from CodeT5+ [31], an advanced pre-trained code model trained on a large corpus of programs using the encoder–decoder architecture, to tokenize the code into a sequence of tokens. After prepending the [CLS] flag at the beginning, the token sequence is fed into the CodeT5+ to extract semantic features. The final hidden states of the tokens in the last layer are then used as the semantic encoding of the code, denoted as h_c.

3.2. Code explanation generation and encoding

FuSEVul explores incorporating code explanation, the textual form description of code semantics, to enhance the model's capability in more effectively comprehending the subtle vulnerability patterns implied within the code. Compared to the code itself, the textual explanation can clearly outline the purpose and functionality of entire functions, intuitively presenting the semantic relationships within the code. Proper integration of such form information thus helps alleviating the difficulty of understanding complex statements and their execution order, aiding the model in better inferring the code logic and improving its understanding.

To ensure high quality explanation be generated for the code, the advanced LLMs that exhibit powerful code comprehension ability, are leveraged. Considering the model ability and price factors, we instruct

Raw Source Code

```
static void good()
{
    char * data;
    char dataBuffer[100] = "";
    data = dataBuffer;
    if(staticFive == 5)
    {
        /* FIX: full path is specified */
        strcpy(data, GOOD_OS_COMMAND);
    }
    {
        FILE *pipe;
        pipe = POPEN(data, "wb");
        if (pipe != NULL)
        {
            PCLOSE(pipe);
        }
    }
}
```

Normalized Code

```
static void FUN1 ( )
{
    char * VAR1 ;
    char VAR2[100] = "";
    VAR1 = VAR3;
    if(VAR4 == 5)
    {
        strcpy(VAR1, VAR5);
    }
    {
        FILE * VAR6;
        VAR6 = FUN2(VAR1, "wb");
        if (VAR6 != NULL)
        {
            FUN3(VAR6);
        }
    }
}
```

Fig. 2. An example of normalization processing on a function from the TrVD dataset.

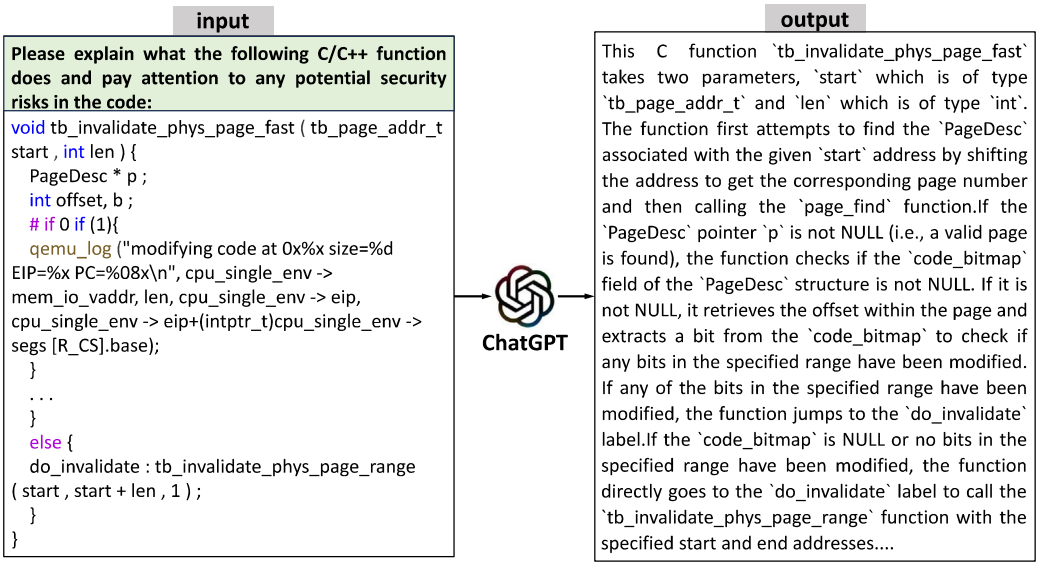

Fig. 3. A prompting example that instruct ChatGPT for code explanation generation.

the OpenAI ChatGPT-3.5 particularly to generate textual explanations regarding both the functionality and potential security aspects of the code. Fig. 3 illustrates the specific prompt template we adopt, where the straightforward zero-shot prompting is leveraged, which can generate satisfactory code explanations. For the LLM-generated textual explanation, the pre-trained RoBERTa [53] model is then utilized to encode it into semantic feature vectors, denoted as v_e.

3.3. Feature fusion and vulnerability detection

In this section, the feature vectors separately extracted from the code and its textual explanation are fused to form a unified and informative vector, enhancing the overall model performance.

Various strategies exist for aggregating vectors from different modalities, with cross-attention [54] mechanism showing superior fusion capabilities. By extending the self-attention [55] mechanism across two distinct vectors, elements from one vector can attend to elements in the other, establishing inter-relationships. Given the high degree of correlation between the code and its explanation, FuSEVul adopts a modified self-attention-based fusion schema to facilitate richer cross-modal interactions, enabling the model to better focus on the relevant aspects of each modality.

Specifically, given the feature vectors $v_c \in \mathbb{R}^{m\times d}$ and $v_e \in \mathbb{R}^{m\times d}$ extracted from the code and its corresponding textual explanation, the devised multi-modal fusion schema of FuSEVul first derives the query, key, and value matrices by multiplying v_c with distinct linear mapping matrices, as formalized below:

$$\mathbf{Q} = v_c\mathbf{W}_q,\ \mathbf{K} = v_c\mathbf{W}_k,\ \mathbf{V} = v_c\mathbf{W}_v \tag{1}$$

Here, $\mathbf{W}_q \in \mathbb{R}^{d\times d_k}$, $\mathbf{W}_k \in \mathbb{R}^{d\times d_k}$, and $\mathbf{W}_v \in \mathbb{R}^{d\times d_v}$ are the trainable weight matrices for the linear mappings. Similarly, a weight matrix $\mathbf{W}_e \in \mathbb{R}^{d\times m}$ is applied to vector v_e, producing a bridging matrix $\mathbf{E} = v_e\mathbf{W}_e$, through which v_e is able to effectively participate in the attention computation with v_c, expressed as:

$$Z = \text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V},\mathbf{E}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathbf{T}}\mathbf{E}}{\sqrt{d_k}}\right)\mathbf{V} \tag{2}$$

A more intuitive illustration of this process of fusing the feature vectors from the two modalities is shown in Fig. 4.

On this basis, the fused vector is fed into a multilayer perceptron (MLP) with two hidden layers, serving as the classification layer for

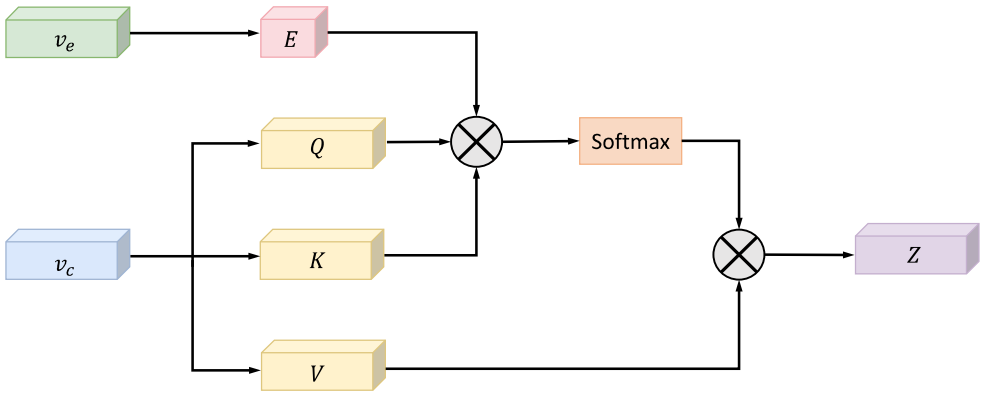

Fig. 4. An illustration of the modified self-attention based multi-modal fusion schema.

vulnerability detection, formally expressed as:

$$\hat{y} = \delta\left(\text{MLP}\left(Z_0\right)\right) \tag{3}$$

where δ represents the softmax function, Z_0 refers to the hidden vector corresponding to the headmost [CLS] token, and $\hat{y}$ denotes the predicted output label. The model's parameters are trained and optimized by minimizing the cross-entropy loss.

4. Experimental setup

4.1. Research questions

In this section, we conduct experiments on multiple widely used benchmark datasets to assess the effectiveness of FuSEVul, addressing the following five *Research Questions* (RQs):

- RQ1: How does FuSEVul perform in vulnerability detection compared to existing methods?
- RQ2: How effective is FuSEVul compared to existing approaches when the training data is limited?
- RQ3: Whether and to what extent does the inclusion of code explanation semantics improve the detection performance of FuSEVul?
- RQ4: How do different fusion strategies influence the performance of FuSEVul?
- RQ5: How does FuSEVul perform when using alternative pre-trained semantic encoders?

4.2. Datasets

To answer these research questions, three widely recognized datasets are used for the evaluation, including Devign [25], Reveal [6], and TrVD [9]. Table 1 provides a concise statistical summary of the datasets, such as the total number of samples, the proportion of vulnerable samples, and the major type of samples in the dataset. Specifically, the Devign dataset is derived from four widely used large-scale C projects (i.e., Linux Kernel, QEMU, Wireshark, and FFmpeg) and was manually annotated through a rigorous two-round labeling and cross-verification process totaling approximately 600 man-hours. It comprises over 27k functions, of which 45.61% are labeled as vulnerable. The Reveal dataset consists of real-world code samples extracted from the Linux Debian Kernel and Chromium. However, it exhibits significant class imbalance, with only 9.85% of its 22k more samples containing vulnerabilities. Unlike these two datasets, the TrVD dataset is sourced from the SARD dataset and primarily contains synthetic samples, which however contains a substantially larger sample quantity of approximately 150k.

Similar as in previous studies, the samples in the datasets are partitioned into training, validation, and test sets at an 8:1:1 ratio. The training set is used to optimize model parameters, the validation set to select the best-performing model, and the test set to assess the final model's performance.

4.3. Baselines

To evaluate the effectiveness of FuSEVul, eighteen advanced DL-based models that span across different paradigms of vulnerability detection are established for a systematic comparison. These baselines include six representative supervised learning based models TokenCNN, TokenRNN, SySeVR, VulCNN, Reveal, and TrVD that operate on typical uni-modal code representations such as code token sequence, code slices, CPG, and AST; two multi-modality fusion models: DeepVD and S^2FVD; eight PLM-based models: UniXcoder, CodeBERT, GraphCodeBERT, CodeT5, CodeT5+, LineVul, EPVD, and CAPTURE; as well as two LLM prompting-based methods: GPT-3.5-Instruct and Codellama-7B. A concise overview of these comparison baselines is as follows:

- **Six supervised learning methods operating on uni-modal data:**
  - **TokenCNN** [15] and **TokenRNN** [56] leverage the TextCNN and BiLSTM respectively to extract indicative features from the code token sequence for vulnerability prediction.
  - **VulCNN** [5] utilizes the program dependence graph (PDG) as the initial code representation. By applying Sent2Vec for node embeddings while quantifying node importance through three graph centrality measures, the PDG is then converted into a three-channel image, on which basis, a CNN model is established for vulnerability detection.
  - **SySeVR** [17] slices the code according to the control and data dependencies on vulnerability-related code elements, such as library API calls and arithmetic expressions. These slices are subsequently processed by a BiGRU to extract discriminative features for detection.
  - **Reveal** [6] operates on the code property graph (CPG), a hybrid graph that merges CFG, AST, and PDG; while a gated graph neural network (GGNN) is established to extract vulnerability-indicative features from this unified representation.
  - **TrVD** [9] decomposes the function's AST into constituent sub-trees, from which a tree-structured recursive neural encoder is leveraged to distill indicative features. After gathering the features from all sub-trees, a Transformer encoder is then utilized to aggregate them into a dense feature vector, with which to classify vulnerability.

Table 1
Statistics of the datasets.

Dataset	#Total	#Vul	#Non-vul	Ratio(%)	Type
Devign	27,318	12,460	14,858	45.61	Real-world
Reveal	22,734	2,240	20,494	9.85	Real-world
TrVD	149,919	55,549	94,370	37.05	Synthetic

- **Two multi-modality fusion models:**
  - **DeepVD** [14] concatenates class-separation features extracted from multiple code representations (token sequence, AST, Post-Dominator Tree (PDT), and Exception Flow Graph (EFG)) using proper neural encoders (GRU, Tree-LSTM and GCN), subsequently combining them for enhanced detection performance.
  - **S^{2}FVD** [57] employs a heterogeneous architecture combining DPCNN (for token sequence), graph attention network (for CFG), and an recursive neural networks (for AST), with an MLP-based fusion layer integrating these modality-specific features.
- **Eight PLM-based models:**
  - **CodeBERT** [28], **GraphCodeBERT** [29], and **UniXcoder** [30] are three widely recognized pre-trained code models adopting the Transformer encoder-only architecture. Here, we utilize their accompanying tokenizers to generate code token sequences for the samples in the datasets, and fine-tune them for vulnerability prediction task.
  - **CodeT5** [58] and **CodeT5+** [31] are also two pre-trained code models but adopts the encoder–decoder architecture. We fine-tune their encoders on the datasets for vulnerability prediction.
  - **LineVul** [7] concatenates the embeddings of BPE sub-tokens with their positional embeddings, and then passed into the CodeBERT model for feature extraction and vulnerability prediction.
  - **EPVD** [33] first constructs and decomposes a syntax-CFG into multiple paths. It then applies CodeBERT and CNN to learn both intra-path and inter-path relationships from these paths, with their extracted feature vectors combined through a MLP layer for classification.
  - **CAPTURE** [34] pioneers an adversarial reprogramming based vulnerability detection method. It transforms the code token sequences into perturbation images by reshaping each token's embedding from a learnable universal perturbation dictionary, on which basis a pre-trained ViT model is reprogrammed by appending a dynamic label remapping scheme that reassigns the model's output to the binary vulnerability detection result.
- **Two LLM Prompting based methods**: **ChatGPT-3.5** [59] and **Codellama-7B** [60] refer to instructing these two LLMs with prompt engineering for vulnerability detection, where we adopt the prompts used in work [43].

4.4. Evaluation metrics

Consistent with most previous DL-based vulnerability detection methods, we use Accuracy (Acc.), Precision (Prec.), Recall (Rec.), and F1 score as primary metrics for performance comparison.

$$Acc = \frac{TP + TN}{TP + FP + TN + FN} \tag{4}$$

$$Prec = \frac{TP}{TP + FP}, \quad Rec = \frac{TP}{TP + FN} \tag{5}$$

$$F1 = 2 \times \frac{Prec \times Rec}{Prec + Rec} \tag{6}$$

Here, TP denotes the number of correctly identified vulnerable samples, FP is the number of benign samples incorrectly classified as vulnerable, TN denotes the number of correctly classified benign samples, and FN is the number of vulnerable samples failed to be detected by the model.

4.5. Implementation details

The pre-trained models used in this study were sourced directly from Hugging Face's model hub. The maximum length of both the code token sequence and the textual explanation sequence is set to 512. Our model was trained for 50 epochs, using a batch size of 16, with the learning rate setting to 2e-6 and the utilizing the Adam optimizer for optimizing the model parameters. The model achieving the highest validation accuracy during training was selected as the final model. All experiments were conducted on a Linux server with two NVIDIA RTX 3090 GPUs.

5. Experimental results

5.1. RQ1: Performance of FuSEVul compared to SOTA

This section evaluates the vulnerability detection performance of FuSEVul in comparison with the baseline methods. As presented in Table 2, FuSEVul consistently surpasses all baseline methods in terms of both accuracy and F1 score across the three evaluated datasets. Specifically, when compared to the six baseline methods that employ typical supervised learning over uni-modal code representations, i.e., TokenCNN, TokenRNN, VulCNN, SyseVR, Reveal, and TrVD, FuSEVul achieves average relative improvements in accuracy of 11.2%, 5.5%, and 3.7%, and in F1 score of 21.1%, 68.3%, and 8.1% on the Devign, Reveal, and TrVD datasets, respectively.

Baseline methods based on pre-trained language models (PLMs), either through fine-tuning or adversarial reprogramming, represent the second-best performing category. This highlights the advantage of utilizing the general code-related knowledge embedded during the PLMs' pre-training phase, which enhances their capacity to capture vulnerability patterns. Nevertheless, FuSEVul demonstrates superior performance, achieving relative gains of 2.7%, 3.0%, and 1.4% in accuracy, and 3.4%, 39.5%, and 8.7% in F1 score across the three datasets, respectively.

The two fusion-based methods, i.e., DeepVD and S^{2}FVD, also exhibit improved performance against the methods that operate on uni-modal code view, attributable to their exploitation of multi-view code features that capture vulnerability patterns from diverse perspectives. However, FuSEVul, by integrating code explanations that provide fundamentally novel and distinct views beyond the direct code-derived features, further improves detection performance. In particular, it achieves relative improvements of 5.8%, 2.4%, and 1.9% in accuracy, and 6.5%, 68.8%, and 2.2% in F1 score, respectively.

Furthermore, much consistent with the observations in prior studies [41,42,61], methods based on large language model (LLM) prompting exhibit substantially lower detection performance compared to the other more specialized technical paradigms. This inferiority can be attributed to the general-purpose nature of LLMs' training objectives and the relative scarcity of vulnerable code examples in the pre-training data, both of which hinder LLMs' capability to accurately recognize vulnerability patterns through prompt engineering alone. On average, FuSEVul achieves significant relative improvements over prompting-based methods, with gains of 12.6%, 11.0%, and 74.0% in accuracy, and 275.1%, 519.3%, and 36.9% in F1 score across the three datasets.

Table 2
Performance comparison with the baseline methods.

Dataset	Devign				Reveal				TrVD			
Method	Acc	F1	Rec	Pre	Acc	F1	Rec	Pre	Acc	F1	Rec	Pre
TokenCNN	54.25	45.27	43.56	47.13	87.42	24.34	27.38	21.90	84.56	80.15	84.14	76.52
TokenRNN	53.26	41.72	45.52	38.50	86.10	26.51	25.91	27.14	83.12	82.68	81.14	84.28
VulCNN	56.17	48.87	45.71	52.49	87.75	33.60	25.67	48.64	88.03	87.21	86.22	88.22
SySeVR	51.84	49.72	44.29	56.66	87.93	25.34	22.29	29.37	87.78	87.72	88.79	86.67
Reveal	54.37	43.48	40.14	47.43	89.07	26.97	24.49	30.00	86.29	79.34	73.88	85.67
TrVD	56.19	49.13	52.64	46.06	83.47	32.37	27.61	39.13	87.88	83.28	85.16	81.48
DeepVD	57.50	51.02	51.15	50.89	89.85	25.59	38.78	19.10	87.61	87.41	89.14	85.74
S^2FVD	56.70	54.05	50.18	**58.57**	89.16	30.18	36.69	25.63	87.81	88.83	82.23	96.57
UniXcoder	57.43	54.96	**61.72**	49.53	90.14	33.44	29.14	39.23	88.22	81.58	70.42	96.94
CodeBERT	57.91	53.82	56.44	51.41	90.54	32.60	24.76	47.70	88.31	81.65	70.22	97.53
GraphCodeBERT	59.63	54.55	55.77	53.39	90.06	35.06	29.04	44.20	88.38	81.77	70.37	**97.58**
CodeT5	58.86	53.78	55.09	52.53	90.05	32.74	26.19	43.65	88.28	81.91	71.62	95.67
CodeT5+	58.96	54.30	56.10	52.60	90.05	38.58	33.80	44.93	88.32	88.78	92.42	84.80
LineVul	59.66	54.24	55.01	53.48	87.68	30.35	29.05	31.77	88.12	82.87	77.60	88.92
EPVD	59.63	53.40	53.56	53.24	87.68	37.22	35.17	39.52	87.84	82.67	87.61	78.25
CAPTURE	58.57	53.53	52.20	54.93	86.32	30.11	28.51	31.90	88.06	81.67	96.75	70.59
ChatGPT-3.5	53.85	22.28	49.72	14.35	81.77	8.91	9.52	8.37	50.86	66.51	**97.58**	50.44
Codellama-7B	53.44	11.20	6.37	46.20	83.49	6.55	5.44	8.23	51.91	64.99	89.30	51.10
FuSEVul	**60.39**	**55.91**	57.79	54.14	**91.68**	**46.76**	**39.52**	**57.24**	**89.41**	**90.02**	95.47	85.16

Answer to RQ1: FuSEVul consistently outperforms all baseline approaches, achieving the most favorable results in terms of both accuracy and F1 score across the three evaluated datasets, thereby confirming its superior effectiveness in vulnerability detection.

5.2. RQ2: Performance under limited training data

The ability to achieve effective training under data-scarce scenarios is considered a critical characteristic of DL-based vulnerability detectors [34], as it enables rapid generalization to newly emerging vulnerability patterns during the early stages of disclosure, when only a limited number of labeled samples are available.

To evaluate the effectiveness of FuSEVul under such conditions, we follow the experimental settings in CAPTURE [34], and assess the model's performance with varying degrees of training data scarcity. Specifically, FuSEVul and all baseline models are configured to be trained with subsets of 10, 50, 100, 500, and 1000 samples randomly drawn from the training sets of the Devign, Reveal, and TrVD datasets. For each setting, the validation and test sets remain unchanged to ensure comparability. To mitigate sampling bias and enhance result reliability, we repeat this process 10 times for each sample size, and report the average performance metrics on the respective test sets.

Fig. 5 presents the results under the varying levels of training data scarcity, highlighting two primary evaluation metrics: accuracy and F1 score. As expected, the performance of FuSEVul and all baseline models generally improves as the number of training samples increases across the Devign, Reveal, and TrVD datasets. This trend reflects the strong dependency of DL-based methods on data volume, confirming the significance of data richness for more effective model learning. Notably, across all evaluated sample sizes and datasets, FuSEVul generally outperforms the competing methods, as indicated by the red lines in the plots. This highlights its robustness and superior generalization ability in low-resource settings.

Specifically, on the Devign dataset, FuSEVul achieves an average F1 score improvement of 5.14% over all baseline models across the data-scarce scenarios. On the Reveal dataset, where training sample size is restricted to only 100, FuSEVul demonstrates dominant performance across all the evaluated data-limited conditions, with an average F1-score improvement of 8.48%, significantly surpassing other models. On the TrVD dataset, FuSEVul also maintains a clear advantage, with an average accuracy gain of 6.84% and an F1 score increase of 16.09% across the different levels of data scarcity. Remarkably, it achieves an F1 score of 85.83% with only 100 training instances, outperforming 13 out of 18 baseline models that were trained using the full training dataset.

Answer to RQ2: FuSEVul exhibits superior effectiveness compared to the baseline models under the evaluated data-scarce scenarios. By providing enriched contextual understanding beyond code syntax alone, the textual code explanations incorporated into FuSEVul make it generalize more effectively with limited training data, alleviating its need for training data volume.

5.3. RQ3: Impact of code explanation to model performance

This section evaluates the contribution of textual code explanations, which are integrated into FuSEVul as a distinct modality of semantic information. To examine whether and to what extent does this additional modality enhances the model's vulnerability detection capability, we conduct an ablation study by removing the explanation component from the model for prediction. As presented in Table 3, the inclusion of textual code explanations helps improve the performance of FuSEVul across all the three evaluated datasets. Specifically, the exclusion of this modality results in absolute decreases in F1 score of 1.61%, 8.18%, and 1.24% on the Devign, Reveal, and TrVD datasets, respectively. These results underscore the active role played by the semantic information conveyed through code explanations, which provides additional contextual cues indicative of subtle flaws or risky behaviors that are not readily captured by the neural encoder from the code alone.

Answer to RQ3: The integration of textual code explanations strengthens the vulnerability detection capability of FuSEVul by complementing the structural information inherent in the code. Ablation results reveal that excluding this modality leads to dataset-specific declines in vulnerability detection performance.

5.4. RQ4: Impact of different fusion strategies

This experiment assesses the effects of the self-attention-based fusion schema, which we introduce in Section 3.3 to more effectively

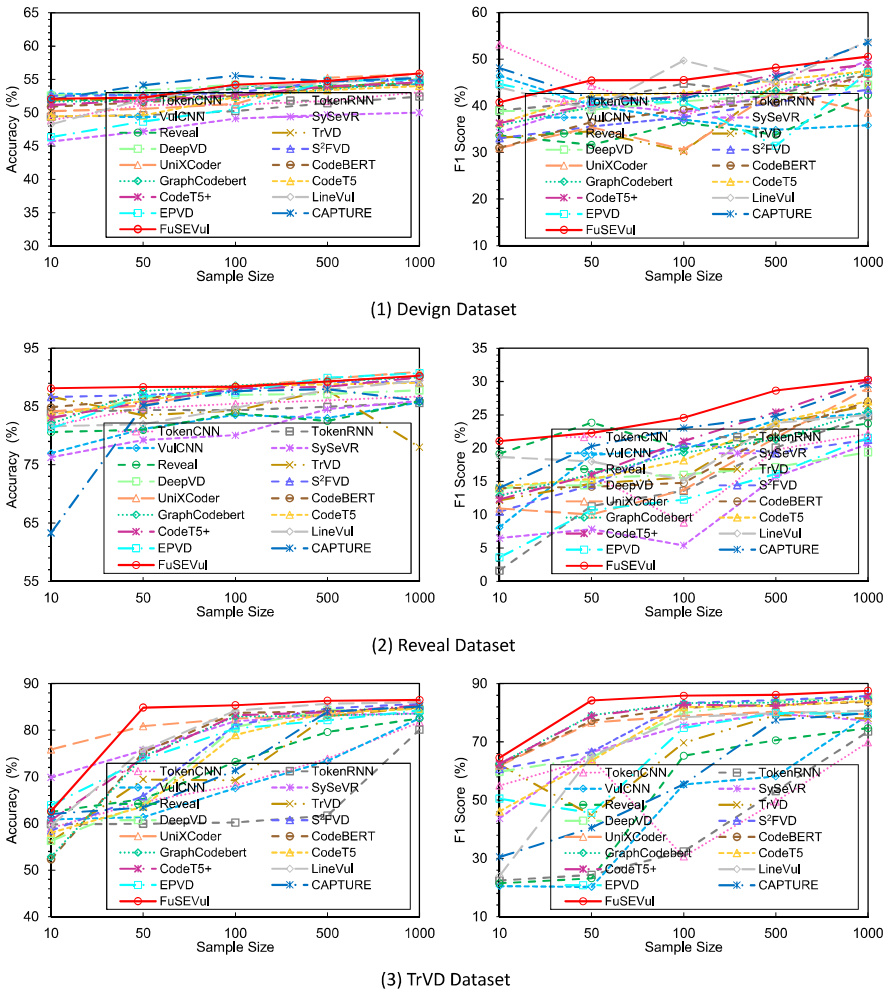

Fig. 5. Comparison of FuSEVul with baseline models under data-limited scenarios.

Table 3
Performance of FuSEVul without the feature contributions from textual code explanations.

Dataset	Method	Acc (%)	F1 (%)	Rec (%)	Pre (%)
Devign	Without Explanation	58.96 (↓ 1.43%)	54.30 (↓ 1.61%)	56.10 (↓ 1.69%)	52.60 (↓ 1.54%)
	FuSEVul	**60.39**	**55.91**	**57.79**	**54.14**
Reveal	Without Explanation	90.05 (↓ 1.63%)	38.58 (↓ 8.18%)	33.80 (↓ 5.72%)	44.93 (↓ 12.31%)
	FuSEVul	**91.68**	**46.76**	**39.52**	**57.24**
TrVD	Without Explanation	88.32 (↓ 1.09%)	88.78 (↓ 1.24%)	92.42 (↓ 3.05%)	84.80 (↓ 0.36%)
	FuSEVul	**89.41**	**90.02**	**95.47**	**85.16**

integrate the heterogeneous features collected from the different modalities. To this end, we replace the original fusion module in FuSEVul with four other commonly adopted fusion strategies, including element-wise addition, concatenation, max-pooling, and cross-attention [54], and compare the performance of FuSEVul with these alternatives across the datasets.

As shown in Fig. 6, FuSEVul, when equipped with the proposed self-attention-based fusion strategy, achieves the best overall performance among all evaluated fusion strategies. It surpasses its variants employing alternative fusion strategies with an absolute improvement of 1.39% in accuracy and 3.04% in F1 score on average across the datasets. The variant utilizing cross-attention ranks second, surpassing the other fusion strategies, which do not incorporate learnable importance weights for modality-specific features.

Furthermore, the performance advantage of the self-attention-based fusion becomes more pronounced on the two real-world datasets. On the Reveal dataset, which features complex and imbalanced distributions of vulnerable versus non-vulnerable samples, FuSEVul achieves an average F1 score gain of 4.94% over the other fusion methods. In contrast, on the TrVD dataset, which comprises relatively simple, synthetic programs, the performance gap is smaller, with an average F1 score improvement of 1.55%. These results indicate that the proposed

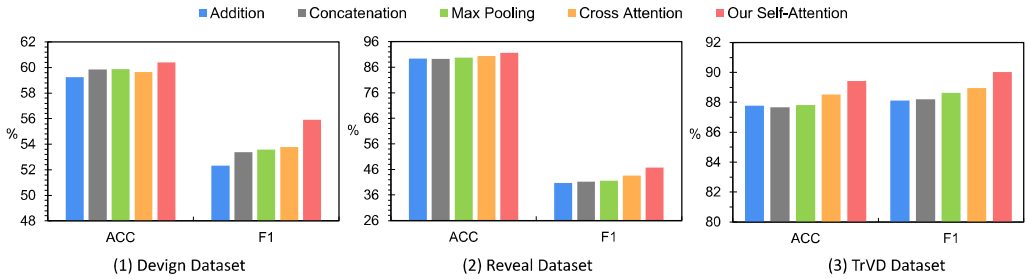

Fig. 6. Comparison of FuSEVul with different fusion methods.

fusion mechanism is more competitive than other fusion strategies, particularly in handling real-world scenarios where code semantics are more intricate and structural variability is higher.

> **Answer to RQ4**: FuSEVul equipped with the devised self-attention-based fusion strategy outperforms four other widely used fusion strategies. It achieves enhanced integration of heterogeneous features collected from the different modalities, yielding superior performance especially when confronted with real-world programs with complex code semantics and structural diversity.

5.5. RQ5: Performance with alternative pre-trained models

In previous experiments, FuSEVul is configured with adopting CodeT5+ as the pre-trained model for code semantic encoding, given its state-of-the-art performance in various program analysis tasks [31]. To assess FuSEVul's robustness and generalizability, we further investigate its performance when equipped with other widely recognized PLMs, including CodeBERT, UniXcoder, GraphCodeBERT, and CodeT5.

As shown in Table 4, FuSEVul maintains competitive performance across the alternative backbone PLMs, with only marginal differences observed. Specifically, the alternative models exhibit averaged relative accuracy degradation of 1.6%, 1.2%, and 0.7%, and F1 score degradation of 0.4%, 6.9%, and 0.7% across the Devign, Reveal, and TrVD datasets, respectively. These small performance gaps suggest that the choice of pre-trained language model exerts limited influence on FuSEVul's overall vulnerability detection capability. This can be attributed to the extensive code understanding capabilities already embedded in these models during large-scale pre-training, which equip them with a foundational comprehension of program semantics.

However, all these variants of FuSEVul that incorporate textual code explanations outperform their uni-modal counterparts, i.e., baseline models fine-tuned using code only, across all three datasets. Particularly, in terms of F1 score, the average relative improvements over the corresponding code-only baselines are 2.7%, 29.0%, and 7.8% on Devign, Reveal, and TrVD, respectively. This again affirm the value of integrating textual explanations as an auxiliary semantic modality, which enhances the model's ability to capture subtle and context-dependent vulnerability patterns that may not be apparent from code alone.

> **Answer to RQ5:** The use of other prominent PLMs as FuSEVul's alternative backbone encoders exhibits only a limited impact on its vulnerability detection performance, with CodeT5+ yielding the best overall results across the majority cases when employed as the backbone encoder.

6. Discussion

Adversarial attacks, such as semantics-preserving code transformations [62–64], have the potential to undermine the reliability of DL-based vulnerability detection methods. However, commonly adopted transformation attacks, such as renaming identifiers [62] and inserting small segments of junk code [65], which have demonstrated effectiveness in misleading models for tasks like code summarization [66] and clone detection [67], are unlikely to affect FuSEVul to a significant degree. This robustness arises from the enforcement of code normalization in FuSEVul, which neutralizes the influence of such superficial modifications before the code is analyzed by the model. Nonetheless, more advanced adversarial strategies, such as planting genuine vulnerabilities into non-malicious samples or vice versa injecting large volumes of unreachable benign code into vulnerable instances may still pose a threat. Exploring alleviating strategies to reinforce FuSEVul against these advanced forms of adversarial manipulation and systematically assessing their potential impact represents a compelling avenue for future work.

In this study, FuSEVul is evaluated on code written in C/C++, the programming languages historically associated with a high prevalence of security vulnerabilities. However, the applicability of the proposed approach, particularly the integration of textual code explanations to enhance the vulnerability understanding of DL models, has yet to be examined in the context of other programming languages, such as Python [68] and emerging languages like Solidity [69,70]. Furthermore, hybrid programming practices, in which multiple programming languages coexist within a single codebase or snippet, are increasingly common in modern software development. The substantial syntactic differences across languages pose significant challenges for vulnerability detection. Nonetheless, our methodology, which incorporates natural language explanations, offers the potential to generalize across programming languages more effectively than approaches that depend solely on source code. Extending FuSEVul to support multilingual vulnerability detection and enabling language-agnostic prediction capabilities remain promising directions for future research.

Given resource constraints, the current implementation of FuSEVul employs ChatGPT-3.5, a high cost-effective LLM, for generating textual code explanations to fuse with. Switching to more recently released LLMs, such as GPT-4.1, Gemini-2.5, and DeepSeek-R1 [71], which generally produce code explanations of higher quality, may further boost the performance of FuSEVul. Moreover, since the primary focus of this work is not on optimizing explanation generation, a zero-prompting strategy is adopted, aligning with recent findings [38] which suggest that advanced prompting methods do not consistently outperform straightforward zero-shot approaches. Nonetheless, the experimental results confirm the efficacy of incorporating natural language explanations to improve vulnerability detection. In future work, contingent on more sufficient computational resources and time availability, we intend to evaluate FuSEVul in conjunction with up-to-date LLMs and more sophisticated prompting strategies, such as Chain-of-Thought and Few-Shot prompting. Moreover, dedicated techniques for mitigating the

Table 4
Performance of FuSEVul with alternative PLMs as backbone code semantic encoders.

Dataset	Devign				Reveal				TrVD			
Method	Acc	F1	Rec	Pre	Acc	F1	Rec	Pre	Acc	F1	Rec	Pre
UniXcoder	58.42 (↓ 1.97%)	56.11 (↑ 0.20%)	61.16 (↑ 3.37%)	51.82 (↓ 2.32%)	90.67 (↓ 1.01%)	41.76 (↓ 5.00%)	36.19 (↓ 3.33%)	49.35 (↓ 7.89%)	88.44 (↓ 0.97%)	89.18 (↓ 0.84%)	95.31 (↓ 0.16%)	83.79 (↓ 1.37%)
GraphCodeBERT	59.74 (↓ 0.65%)	55.32 (↓ 0.59%)	57.37 (↓ 0.42%)	53.41 (↓ 0.73%)	90.37 (↓ 1.31%)	45.39 (↓ 1.37%)	43.33 (↑ 3.81%)	47.64 (↓ 9.60%)	88.91 (↓ 0.50%)	89.30 (↓ 0.72%)	92.58 (↓ 2.89%)	86.24 (↑ 1.08%)
CodeT5	59.30 (↓ 1.09%)	55.52 (↓ 0.39%)	58.47 (↑ 0.68%)	52.86 (↓ 1.28%)	90.23 (↓ 1.45%)	41.58 (↓ 5.18%)	37.62 (↓ 1.90%)	46.47 (↓ 10.77%)	88.71 (↓ 0.70%)	89.35 (↓ 0.67%)	94.69 (↓ 0.78%)	84.58 (↓ 0.58%)
CodeBERT	60.29 (↓ 0.10%)	55.88 (↓ 0.03%)	57.88 (↑ 0.09%)	54.01 (↓ 0.13%)	91.06 (↓ 0.62%)	46.15 (↓ 0.61%)	41.43 (↑ 1.91%)	52.09 (↓ 5.15%)	89.14 (↓ 0.27%)	89.88 (↓ 0.14%)	96.41 (↑ 0.94%)	84.17 (↓ 0.99%)
CodeT5+	60.39	55.91	57.79	54.14	91.68	46.76	39.52	57.24	89.41	90.02	95.47	85.16

potential hallucinations of LLMs [72–74] may be integrated to enhance the reliability and quality of the generated code explanations.

As a deep learning-based method, FuSEVul is inherently constrained by the interpretability challenges associated with its binary detection outcomes, a limitation that is commonly observed in many other DL-based vulnerability detectors. The absence of clear, interpretable explanations for the model's predictions may undermine its trustworthiness, particularly in real-world applications where transparency is essential for user confidence. To mitigate this limitation, it is promising to integrate general explainable AI techniques [23,75,76] to perform post-event explanation. Alternatively, a more direct approach could involve designing fine-grained detection models [77–80] that provide insights into the specific statements or code paths critical to the predictions, thereby elucidating the rationale behind the model's outputs. For FuSEVul, this would require generating explanations at the statement or block level and developing appropriate mechanisms to incorporate these fine-grained explanations into the detection process, ultimately improving detection accuracy and interpretability. We leave the exploration of enhancing FuSEVul's interpretability as an important avenue for future research.

7. Conclusion

This work present a multimodal fusion-based vulnerability prediction method called FuSEVul, which pioneers the integration of code semantics with LLM-generated natural language explanations to better identify latent vulnerability patterns. By combining a pre-trained language model for encoding structural and semantic information from source code, an LLM-based module for generating risk-aware functional descriptions, and a self-attention-driven fusion mechanism for aligning and integrating cross-modal features, FuSEVul achieves a more holistic understanding of program behavior and security flaws. Extensive experiments conducted on three public benchmarks demonstrate the superiority of FuSEVul, with average relative improvements of 7.3% in accuracy and 52.2% in F1 score compared to state-of-the-art baselines. The incorporation of textual explanations enhances the contextual understanding of code, FuSEVul exhibits an absolute F1 gain of 9.59% over the baseline models in data-scarce scenarios, confirming its robustness under limited supervision. Ablation studies further underscore the critical role of both the LLM-generated explanations and the fusion strategy, whose removal results in varying degrees of performance degradation across the datasets. Future research directions include enhancing the model's resilience to adversarial attacks, extending its applicability to multilingual and language-agnostic settings, and improving the quality and informativeness of the generated explanations through the use of alternative LLMs and more advanced prompting techniques.

CRediT authorship contribution statement

Zhenzhou Tian: Methodology, Supervision, Conceptualization. **Minghao Li:** Data curation, Software, Writing – original draft. **Jiaze Sun:** Investigation, Methodology. **Yanping Chen:** Validation, Conceptualization, Supervision. **Lingwei Chen:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Zhenzhou Tian reports financial support was provided by National Natural Science foundation of china. Zhenzhou Tian reports financial support was provided by Natural Science Basic Research Program of Shaanxi Province. Minghao Li reports financial support was provided by Graduate Innovation Fund of Xi'an University of Posts and Telecommunications.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (62272387, 61702414), the Natural Science Basic Research Program of Shaanxi (2022JM-342, 2018JQ6078), and the Graduate Innovation Fund of Xi'an University of Posts and Telecommunications, China (CXJJYL2024038).

Data availability

I have shared the link to my data and code in the manuscript.

References

[1] CVE, 2024. https://cve.mitre.org/.
[2] IBM, Cost of a data breach report, 2023, https://www.ibm.com/reports/data-breach.
[3] Z. Tian, Y. Teng, X. Ke, Y. Chen, L. Chen, SolBERT: Advancing solidity smart contract similarity analysis via self-supervised pre-training and contrastive fine-tuning, Inf. Softw. Technol. 184 (2025) 107766, http://dx.doi.org/10.1016/j.infsof.2025.107766, URL https://www.sciencedirect.com/science/article/pii/S0950584925001053.
[4] N. Shiri Harzevili, A. Boaye Belle, J. Wang, S. Wang, Z.M.J. Jiang, N. Nagappan, A systematic literature review on automated software vulnerability detection using machine learning, ACM Comput. Surv. 57 (3) (2024) http://dx.doi.org/10.1145/3699711.
[5] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, H. Jin, VulCNN: an image-inspired scalable vulnerability detection system, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 2365–2376, http://dx.doi.org/10.1145/3510003.3510229.
[6] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2022) 3280–3296.
[7] M. Fu, C. Tantithamthavorn, LineVul: a transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 608–620, http://dx.doi.org/10.1145/3524842.3528452.
[8] H. Hanif, S. Maffeis, VulBERTa: Simplified source code pre-training for vulnerability detection, in: 2022 International Joint Conference on Neural Networks, IJCNN, 2022, pp. 1–8, http://dx.doi.org/10.1109/IJCNN55064.2022.9892280.

[9] Z. Tian, B. Tian, J. Lv, Y. Chen, L. Chen, Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding, Expert Syst. Appl. 238 (2024) 121865, http://dx.doi.org/10.1016/j.eswa.2023.121865, URL https://www.sciencedirect.com/science/article/pii/S0957417423023679.

[10] Y. Huang, M. He, X. Wang, J. Zhang, HeVulD: A static vulnerability detection method using heterogeneous graph code representation, Trans. Info. for. Sec. 19 (2024) 9129–9144, http://dx.doi.org/10.1109/TIFS.2024.3457162.

[11] S. Li, L. Jiang, Q. Zhang, Z. Wang, Z. Tian, M. Guizani, A malicious mining code detection method based on multi-features fusion, IEEE Trans. Netw. Sci. Eng. 10 (5) (2023) 2731–2739, http://dx.doi.org/10.1109/TNSE.2022.3155187.

[12] J. Fan, Y. He, H. Wu, Small sample smart contract vulnerability detection method based on multi-layer feature fusion, Complex & Intell. Syst. 11 (4) (2025) 198, http://dx.doi.org/10.1007/s40747-025-01782-3.

[13] X. Li, L. Liu, Y. Liu, H. Liu, Detecting android malware: A multimodal fusion method with fine-grained feature, Inf. Fusion 114 (2025) 102662, http://dx.doi.org/10.1016/j.inffus.2024.102662, URL https://www.sciencedirect.com/science/article/pii/S1566253524004408.

[14] W. Wang, T.N. Nguyen, S. Wang, Y. Li, J. Zhang, A. Yadavally, DeepVD: Toward class-separation features for neural network vulnerability detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, 2023, pp. 2249–2261, http://dx.doi.org/10.1109/ICSE48619.2023.00189.

[15] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, M. McConley, Automated vulnerability detection in source code using deep representation learning, in: 2018 17th IEEE International Conference on Machine Learning and Applications, ICMLA, 2018, pp. 757–762, http://dx.doi.org/10.1109/ICMLA.2018.00120.

[16] K. Napier, T. Bhowmik, S. Wang, An empirical study of text-based machine learning models for vulnerability detection, Empir. Softw. Eng. 28 (2) (2023) 38, http://dx.doi.org/10.1007/s10664-022-10276-6.

[17] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2244–2258, http://dx.doi.org/10.1109/TDSC.2021.3051525.

[18] D. Zou, S. Wang, S. Xu, Z. Li, H. Jin, muVulDeePecker: A deep learning-based system for multiclass vulnerability detection, TDSC (2021) 2224–2236, http://dx.doi.org/10.1109/TDSC.2019.2942930.

[19] Z. Tang, Q. Hu, Y. Hu, W. Kuang, J. Chen, SEVulDet: A semantics-enhanced learnable vulnerability detector, in: 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN, 2022, pp. 150–162, http://dx.doi.org/10.1109/DSN53405.2022.00026.

[20] L. Wang, G. Lu, X. Chen, X. Dai, J. Qiu, SIFT: enhance the performance of vulnerability detection by incorporating structural knowledge and multi-task learning, Autom. Softw. Eng. 32 (2) (2025) 38, http://dx.doi.org/10.1007/s10515-025-00507-7.

[21] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J.M. Zhang, Q. Liao, Vulnerability detection with graph simplification and enhanced graph representation learning, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, 2023, pp. 2275–2286, http://dx.doi.org/10.1109/ICSE48619.2023.00191.

[22] S. Cao, X. Sun, X. Wu, D. Lo, L. Bo, B. Li, W. Liu, Coca: Improving and explaining graph neural network-based vulnerability detection systems, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024, http://dx.doi.org/10.1145/3597503.3639168.

[23] Z. Chu, Y. Wan, Q. Li, Y. Wu, H. Zhang, Y. Sui, G. Xu, H. Jin, Graph neural networks for vulnerability detection: A counterfactual explanation, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, in: ISSTA 2024, Association for Computing Machinery, New York, NY, USA, 2024, pp. 389–401, http://dx.doi.org/10.1145/3650212.3652136.

[24] Y. Li, et al., Vulnerability detection with fine-grained interpretations, in: ESEC/FSE, 2021, pp. 292–303.

[25] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 32, Curran Associates, Inc., 2019, URL https://proceedings.neurips.cc/paper_files/paper/2019/file/49265d2447bc3bbfe9e76306ce40a31f-Paper.pdf.

[26] X.-C. Wen, C. Gao, J. Ye, Y. Li, Z. Tian, Y. Jia, X. Wang, Meta-path based attentional graph learning model for vulnerability detection, IEEE Trans. Softw. Eng. 50 (3) (2024) 360–375, http://dx.doi.org/10.1109/TSE.2023.3340267.

[27] W. Cai, J. Chen, J. Yu, W. Hu, L. Gao, A software vulnerability detection method based on multi-modality with unified processing, Inf. Softw. Technol. 182 (2025) 107703, http://dx.doi.org/10.1016/j.infsof.2025.107703, URL https://www.sciencedirect.com/science/article/pii/S0950584925000424.

[28] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, 2020, pp. 1536–1547, Online.

[29] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S.K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, M. Zhou, GraphCode{bert}: Pre-training code representations with data flow, in: International Conference on Learning Representations, 2021, URL https://openreview.net/forum?id=jLoC4ez43PZ.

[30] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, J. Yin, UniXcoder: Unified cross-modal pre-training for code representation, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Dublin, Ireland, 2022, pp. 7212–7225.

[31] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, S. Hoi, CodeT5+: Open code large language models for code understanding and generation, in: H. Bouamor, J. Pino, K. Bali (Eds.), Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Singapore, 2023, pp. 1069–1088, http://dx.doi.org/10.18653/v1/2023.emnlp-main.68, URL https://aclanthology.org/2023.emnlp-main.68/.

[32] Z. Liu, Z. Tang, J. Zhang, X. Xia, X. Yang, Pre-training by predicting program dependencies for vulnerability analysis tasks, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024, http://dx.doi.org/10.1145/3597503.3639142.

[33] J. Zhang, Z. Liu, X. Hu, X. Xia, S. Li, Vulnerability detection by learning from syntax-based execution paths of code, IEEE Trans. Softw. Eng. 49 (8) (2023) 4196–4212, http://dx.doi.org/10.1109/TSE.2023.3286586.

[34] Z. Tian, R. Qiu, Y. Teng, J. Sun, Y. Chen, L. Chen, Towards cost-efficient vulnerability detection with cross-modal adversarial reprogramming, J. Syst. Softw. 223 (2025) 112365, http://dx.doi.org/10.1016/j.jss.2025.112365, URL https://www.sciencedirect.com/science/article/pii/S0164121225000330.

[35] Z. Tian, H. Li, H. Sun, Y. Chen, L. Chen, HardVD: High-capacity cross-modal adversarial reprogramming for data-efficient vulnerability detection, Inform. Sci. 686 (2025) 121370.

[36] R. Wang, S. Xu, Y. Tian, X. Ji, X. Sun, S. Jiang, SCL-CVD: Supervised contrastive learning for code vulnerability detection via GraphCodeBERT, Comput. Secur. 145 (C) (2024) http://dx.doi.org/10.1016/j.cose.2024.103994.

[37] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, S.K. Lahiri, LLM-based test-driven interactive code generation: User study and empirical evaluation, IEEE Trans. Softw. Eng. 50 (9) (2024) 2254–2268, http://dx.doi.org/10.1109/TSE.2024.3428972.

[38] W. Sun, Y. Miao, Y. Li, H. Zhang, C. Fang, Y. Liu, G. Deng, Y. Liu, Z. Chen, Source Code Summarization in the Era of Large Language Models , in: 2025 IEEE/ACM 47th International Conference on Software Engineering, ICSE, IEEE Computer Society, Los Alamitos, CA, USA, 2025, pp. 419–431, http://dx.doi.org/10.1109/ICSE55347.2025.00034, URL https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00034.

[39] Y. Luo, R. Yu, F. Zhang, L. Liang, Y. Xiong, Bridging gaps in LLM code translation: Reducing errors with call graphs and bridged debuggers, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 2448–2449, http://dx.doi.org/10.1145/3691620.3695322.

[40] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we? in: 2025 IEEE/ACM 47th International Conference on Software Engineering, ICSE, IEEE Computer Society, Los Alamitos, CA, USA, 2025, pp. 469–481, http://dx.doi.org/10.1109/ICSE55347.2025.00038, URL https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00038.

[41] X. Yin, C. Ni, S. Wang, Multitask-based evaluation of open-source LLM on software vulnerability, IEEE Trans. Softw. Eng. 50 (11) (2024) 3071–3087, http://dx.doi.org/10.1109/TSE.2024.3470333, URL https://doi.ieeecomputersociety.org/10.1109/TSE.2024.3470333.

[42] Y. Guo, C. Patsakis, Q. Hu, Q. Tang, F. Casino, Outside the comfort zone: Analysing LLM capabilities in software vulnerability detection, in: J. Garcia-Alfaro, R. Kozik, M. Choraś, S. Katsikas (Eds.), Computer Security – ESORICS 2024, Springer Nature Switzerland, Cham, 2024, pp. 271–289.

[43] M. Fu, C.K. Tantithamthavorn, V. Nguyen, T. Le, ChatGPT for vulnerability detection, classification, and repair: How far are we? in: 2023 30th Asia-Pacific Software Engineering Conference, APSEC, 2023, pp. 632–636, http://dx.doi.org/10.1109/APSEC60848.2023.00085.

[44] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning, J. Syst. Softw. 212 (C) (2024) http://dx.doi.org/10.1016/j.jss.2024.112031.

[45] X. Zhou, T. Zhang, D. Lo, Large language model for vulnerability detection: Emerging results and future directions, in: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, in: ICSE-NIER'24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 47–51, http://dx.doi.org/10.1145/3639476.3639762.

[46] B. Steenhoek, M.M. Rahman, M.K. Roy, M.S. Alam, H. Tong, S. Das, E.T. Barr, W. Le, To err is machine: Vulnerability detection challenges LLM reasoning, 2025, arXiv:2403.17218, URL https://arxiv.org/abs/2403.17218.

[47] N. Capuano, V. Carletti, P. Foggia, G. Parrella, M. Vento, Leveraging open-source LLMs for zero-shot vulnerability detection: A comparative analysis, in: L. Barolli (Ed.), Advanced Information Networking and Applications, Springer Nature Switzerland, Cham, 2025, pp. 13–25.
[48] Z. Zheng, K. Ning, Q. Zhong, J. Chen, W. Chen, L. Guo, W. Wang, Y. Wang, Towards an understanding of large language models in software engineering tasks, Empir. Softw. Eng. 30 (2) (2024) 50, http://dx.doi.org/10.1007/s10664-024-10602-0.
[49] B. Steenhoek, M.M. Rahman, M.K. Roy, M.S. Alam, E.T. Barr, W. Le, A comprehensive study of the capabilities of large language models for vulnerability detection, 2024, ArXiv, abs/2403.17218, URL https://api.semanticscholar.org/CorpusID:268691784.
[50] Y. Li, X. Li, H. Wu, M. Xu, Y. Zhang, X. Cheng, F. Xu, S. Zhong, Everything you wanted to know about LLM-based vulnerability detection but were afraid to ask, 2025, arXiv:2504.13474, URL https://arxiv.org/abs/2504.13474.
[51] A. Zibaeirad, M. Vieira, Reasoning with LLMs for zero-shot vulnerability detection, 2025, arXiv:2503.17885, URL https://arxiv.org/abs/2503.17885.
[52] X. Du, G. Zheng, K. Wang, J. Feng, W. Deng, M. Liu, B. Chen, X. Peng, T. Ma, Y. Lou, Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG, 2024, arXiv:2406.11147, URL https://arxiv.org/abs/2406.11147.
[53] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: A robustly optimized BERT pretraining approach, 2019, ArXiv, abs/1907.11692, URL https://api.semanticscholar.org/CorpusID:198953378.
[54] H. Li, X.-J. Wu, CrossFuse: A novel cross attention mechanism based infrared and visible image fusion approach, Inf. Fusion 103 (2024) 102147, http://dx.doi.org/10.1016/j.inffus.2023.102147, URL https://www.sciencedirect.com/science/article/pii/S1566253523004633.
[55] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: I. Guyon, U.V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, R. Garnett (Eds.), in: Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017, URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
[56] M. Schuster, K.K. Paliwal, Bidirectional recurrent neural networks, IEEE Trans. Signal Process. 45 (11) (1997) 2673–2681.
[57] Z. Tian, B. Tian, J. Lv, L. Chen, Learning and fusing multi-view code representations for function vulnerability detection, Electronics 12 (11) (2023).
[58] Y. Wang, W. Wang, S. Joty, S.C. Hoi, CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: M.-F. Moens, X. Huang, L. Specia, S.W.-t. Yih (Eds.), Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 8696–8708, http://dx.doi.org/10.18653/v1/2021.emnlp-main.685, URL https://aclanthology.org/2021.emnlp-main.685/.
[59] ChatGPT, Chatgpt, 2022, https://chat.openai.com/.
[60] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X.E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C.C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, G. Synnaeve, Code llama: Open foundation models for code, 2024, arXiv:2308.12950, URL https://arxiv.org/abs/2308.12950.
[61] T. Zhang, C. Yang, Y. Su, M. Weyssow, H. Nguyen, T. Bui, H.J. Kang, Y. Li, E.L. Ouh, L.K. Shar, D. Lo, Benchmarking large language models for multi-language software vulnerability detection, 2025, arXiv:2503.01449, URL https://arxiv.org/abs/2503.01449.
[62] Y. Yang, H. Fan, C. Lin, Q. Li, Z. Zhao, C. Shen, Exploiting the adversarial example vulnerability of transfer learning of source code, IEEE Trans. Inf. Forensics Secur. 19 (2024) 5880–5894, http://dx.doi.org/10.1109/TIFS.2024.3402153.
[63] D. Liu, S. Zhang, ALANCA: Active learning guided adversarial attacks for code comprehension on diverse pre-trained and large language models, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, 2024, pp. 602–613, http://dx.doi.org/10.1109/SANER60148.2024.00067.
[64] Y. Qu, S. Huang, Y. Yao, A survey on robustness attacks for deep code models, Autom. Softw. Eng. 31 (2) (2024) 65, http://dx.doi.org/10.1007/s10515-024-00464-7.
[65] Y. Li, S. Qi, C. Gao, Y. Peng, D. Lo, M.R. Lyu, Z. Xu, Understanding the robustness of transformer-based code intelligence via code transformation: Challenges and opportunities, IEEE Trans. Softw. Eng. 51 (2) (2025) 521–547, http://dx.doi.org/10.1109/TSE.2024.3524461.
[66] A. Jha, C.K. Reddy, CodeAttack: Code-based adversarial attacks for pre-trained programming language models, AAAI 37 (12) (2023) 14892–14900, http://dx.doi.org/10.1609/aaai.v37i12.26739.
[67] H. Zhang, S. Lu, Z. Li, Z. Jin, L. Ma, Y. Liu, G. Li, CodeBERT-Attack: Adversarial attack against source code deep learning models via pre-trained model, J. Softw.: Evol. Process. 36 (3) (2024) e2571, http://dx.doi.org/10.1002/smr.2571, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2571, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.2571.
[68] A. Mechri, M.A. Ferrag, M. Debbah, SecureQwen: Leveraging LLMs for vulnerability detection in python codebases, Comput. Secur. 148 (2025) 104151, http://dx.doi.org/10.1016/j.cose.2024.104151, URL https://www.sciencedirect.com/science/article/pii/S0167404824004565.
[69] T. Wang, X. Zhao, J. Zhang, TMF-net: Multimodal smart contract vulnerability detection based on multiscale transformer fusion, Inf. Fusion 122 (2025) 103189, http://dx.doi.org/10.1016/j.inffus.2025.103189, URL https://www.sciencedirect.com/science/article/pii/S1566253525002623.
[70] J. Shang, J. Li, Y. Sui, H. Guo, X. Gao, D. Zhang, Y. Guo, G. Wu, CEGT: Smart contract vulnerability detection via connectivity-enhanced GCN-transformer, J. Syst. Softw. 227 (2025) 112454, http://dx.doi.org/10.1016/j.jss.2025.112454, URL https://www.sciencedirect.com/science/article/pii/S0164121225001220.
[71] DeepSeek-AI, D. Guo, D. Yang, et al., DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025, arXiv:2501.12948, URL https://arxiv.org/abs/2501.12948.
[72] J. Ma, Z. Gao, Q. Chai, W. Sun, P. Wang, H. Pei, J. Tao, L. Song, J. Liu, C. Zhang, L. Cui, Debate on graph: A flexible and reliable reasoning framework for large language models, Proc. the AAAI Conf. Artif. Intell. 39 (23) (2025) 24768–24776, http://dx.doi.org/10.1609/aaai.v39i23.34658, URL https://ojs.aaai.org/index.php/AAAI/article/view/34658.
[73] J. Ma, N. Qu, Z. Gao, R. Xing, J. Liu, H. Pei, J. Xie, L. Song, P. Wang, J. Tao, Z. Su, Deliberation on priors: Trustworthy reasoning of large language models on knowledge graphs, 2025, arXiv:2505.15210, URL https://arxiv.org/abs/2505.15210.
[74] J. Ma, P. Wang, D. Kong, Z. Wang, J. Liu, H. Pei, J. Zhao, Robust visual question answering: Datasets, methods, and future challenges, IEEE Trans. Pattern Anal. Mach. Intell. 46 (8) (2024) 5575–5594, http://dx.doi.org/10.1109/TPAMI.2024.3366154.
[75] B. Cheng, S. Zhao, K. Wang, M. Wang, G. Bai, R. Feng, Y. Guo, L. Ma, H. Wang, Beyond fidelity: Explaining vulnerability localization of learning-based detectors, ACM Trans. Softw. Eng. Methodol. 33 (5) (2024) http://dx.doi.org/10.1145/3641543.
[76] A. Bennetot, I. Donadello, A. El Qadi El Haouari, M. Dragoni, T. Frossard, B. Wagner, A. Sarranti, S. Tulli, M. Trocan, R. Chatila, A. Holzinger, A. Davila Garcez, N. Díaz-Rodríguez, A practical tutorial on explainable AI techniques, ACM Comput. Surv. 57 (2) (2024) http://dx.doi.org/10.1145/3670685.
[77] G. Tang, L. Yang, L. Zhang, H. Kuang, H. Wang, MRC-VulLoc: Software source code vulnerability localization based on multi-choice reading comprehension, Comput. Secur. 141 (2024) 103816, http://dx.doi.org/10.1016/j.cose.2024.103816.
[78] Y. Ding, S. Suneja, Y. Zheng, J. Laredo, A. Morari, G. Kaiser, B. Ray, VELVET: a novel ensemble learning approach to automatically locate VulnErable statements, in: SANER, 2022, pp. 959–970.
[79] X. Cheng, X. Nie, N. Li, H. Wang, Z. Zheng, Y. Sui, How about bug-triggering paths?-understanding and characterizing learning-based vulnerability detectors, TDSC 21 (2) (2024) 542–558.
[80] W. Tao, X. Su, Y. Ke, Y. Han, Y. Zheng, H. Wei, Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture, Knowl.-Based Syst. 316 (2025) 113341, http://dx.doi.org/10.1016/j.knosys.2025.113341.